



CurriCraft AI (Unified AICTE Model Curriculum Portal)

CurriCraft AI is envisioned as an end-to-end platform for collaboratively drafting, validating, and publishing technical curricula in compliance with NEP 2020 guidelines. Unlike traditional workflows of emailing static documents and manual reviews, CurriCraft uses **real-time CRDT-based co-editing**, **Git-style version control**, **automated NEP-compliance checks**, and **AI-assisted content generation**. This ensures committees can work concurrently on the same syllabus, track changes via commits and diff views, and automatically enforce credit limits and mandated components (e.g. universal human values courses, internships) [26†L21-L29] [33†L3465-L3473]. The portal's architecture combines a modern MERN (or Java Spring) stack with WebSockets, Redis queues, and AI/NLP services to achieve high responsiveness, scalability, and rich functionality.

1. Executive Summary

CurriCraft AI transforms curriculum development into a dynamic, collaborative process. Committees can **co-author syllabi live** (much like Google Docs) using a rich-text editor with conflict-free CRDT syncing [57†L112-L120] [42†L210-L218]. All edits generate immutable “commit” snapshots (e.g. `v2026.1.0-draft`), enabling branches and merge-requests akin to Git. A built-in compliance engine continuously checks drafts against NEP 2020 rules – for example, ensuring an undergraduate program has ≤ 160 total credits [16†L1455-L1459] and includes required components like human values courses and mandatory internships [33†L3465-L3473]. AI modules assist by generating course module suggestions and linking open educational resources. The result is a **modern, fully traceable workflow**: simultaneous editing, automated policy checks, and AI recommendations, leading to faster, more consistent model curricula aligned with national standards [26†L21-L29] [57†L159-L163].

2. Core System Architecture & Tech Stack

The system is a responsive web application with real-time collaboration, built on proven web technologies:

- **Frontend:** A React (TypeScript) single-page app using Tailwind CSS. For rich-text editing, we integrate **TipTap** (a React wrapper around ProseMirror) with **Yjs**, a CRDT-based library. This combination provides a Google-Docs-like editor with live cursors and presence indicators [35†L83-L92] [57†L112-L120]. State can be managed with Redux Toolkit or Zustand, and complex UI elements (charts, visuals) with Recharts or Chart.js.
- **Backend:** Either a Node.js (Express) server or Java Spring Boot. It exposes REST APIs and a WebSocket (WS) server for live collaboration. Socket.IO (Node.js) or Spring's STOMP/WebSocket support will broadcast CRDT updates. Each client's Yjs “Y.Doc” emits changes that are sent to peers; these CRDT updates are **commutative and idempotent**, meaning they can arrive in any order and still converge to the same document state [38†L917-L924] [57†L112-L120].

- **Real-Time Engine:** The heart of live editing is the Yjs document state replicated across clients. We run a lightweight collaboration server (e.g. [y-websocket](#) or a custom Socket.IO provider) that relays Yjs updates. Yjs also provides an *awareness* layer for sharing cursors and user info [57†L126-L134] . For example, a `TiptapCollabProvider` can connect to the WS server and sync the Y.Doc. To support section locking, the UI can optionally disable editing on a module if a user “locks” it; under the hood CRDT still merges changes, but the client respects lock status for UX.
- **Background Jobs:** Heavy or asynchronous tasks (PDF export, analytics, compliance scanning) are offloaded to a job queue. We can use **BullMQ** on top of Redis for Node.js (or Spring Boot’s thread pools). A job queue architecture (“HTTP app → Redis queue → worker” [44†L52-L60]) ensures that requests return immediately while workers process data in the background (with retries, backoff, concurrency, etc. [44†L52-L60] [44†L61-L64]).
- **Database & Storage:**
 - **MongoDB** is the primary database. Its flexible document schema suits the nested structure of curricula (degrees, modules, outcomes). For example, each “curriculum” document contains arrays of modules and resources (see data models below). MongoDB also stores version snapshots, user data, and access logs.
 - **Redis** serves as a fast cache and coordination layer. For instance, Redis can store the current active Yjs document state for a curriculum so new collaborators see the latest content quickly, and maintain pub/sub channels or socket room state for collaboration presence. Redis also holds session data (if using token sessions), rate-limiting counters, and job queue state.
 - **File Storage:** AWS S3 (or self-hosted MinIO) holds large binary assets: reference textbooks, multimedia lecture notes, and exported PDFs. The API will stream these from S3 URLs.
 - **Versioning:** To guard against tampering, every published curriculum version can include a SHA-256 hash of its content. This way, the system can detect accidental corruption or manual edits to stored JSON (a practice similar to how Git uses hashes to verify object integrity).
- **Security & Governance:**
 - **Authentication:** OAuth 2.0 (e.g. via Google/Microsoft accounts) or a custom identity provider issues JWTs for API access. We implement refresh token rotation and short-lived access tokens as a best practice [44†L52-L60] .
 - **Authorization:** A hierarchical **RBAC** model is used. Roles include AICTE Admin, Bureau Head, Curriculum Expert, and Public User. We can support **role hierarchies** so higher roles inherit lower roles’ permissions [54†L93-L101] (for example, a Bureau Head may have all rights of a Curriculum Expert plus additional privileges). An inline authorization library (or Spring Security with method-level guards) enforces these roles on each endpoint [54†L93-L101] [54†L53-L61] .
 - **Data Integrity & Audit:** All commits and changes include metadata (author, timestamp) and are immutable. We log all user actions for audit trails. When a curriculum is published, its JSON is hashed with SHA-256 and stored; any subsequent download of that version can be verified against the hash to ensure integrity.

3. Comprehensive Breakdown of Unique Features

3.1 Real-Time Collaborative Canvas (Google Docs for Syllabi)

- **Multi-User Editing:** Using React and TipTap+Yjs, multiple experts can edit the same curriculum document **simultaneously** with no locking needed [57†L112-L120] . Each user's actions (inserting text, reordering sections, adding outcomes) generate Yjs updates that the server broadcasts to others. Because Yjs is CRDT-based, concurrent edits automatically merge without conflicts [57†L112-L120] . (In contrast to naive locking, our approach provides *eventual consistency* – each participant's view will converge to the same final state [37†L139-L147] [57†L112-L120] .)
- **Live Presence & Cursors:** The editor displays who else is online and where they are editing. Yjs's "awareness" API shares ephemeral data (user names, cursor positions, text selections) among clients [57†L126-L134] . This gives users context (seeing collaborators' cursors and highlights in real time) and improves coordination, similar to Google Docs' presence indicators [57†L126-L134] .
- **Section-Level Locking:** For sensitive content, a Bureau Head might "lock" a module so only a designated expert can edit it, while others can still comment. Technically, this is a UI feature: when locked, other users' editors disable input on that section. The CRDT model isn't preempted – it simply ensures that any attempted edits (if any) do not disturb the locked content. Section locking supplements the CRDT by giving structured control over who can change critical parts.

3.2 Git-Style Branching, Merging, and Version Control

- **Commit & Tag System:** Every time the content is submitted (or auto-saved), the backend captures a **snapshot** of the entire curriculum JSON in a Version collection (see data models below). This snapshot gets a unique `commitHash` (e.g. SHA-1 or UUID) and metadata (author, message). Like Git, we maintain a linked list of commits via `parentHash` fields, forming a directed acyclic commit graph. For example, publishing a revision might tag it `v2026.1.0-rc1`.
- **Visual Diff Viewer:** The UI provides side-by-side comparisons of any two commits. We can use a JSON diff library (or even text-based diff on the serialized syllabus) to highlight added/removed topics, learning outcomes, or credit changes. For instance, if "Data Structures" learning outcomes were added, the diff would color-code them. This mimics versioning tools (e.g. Git diffs) but specialized for curriculum JSON.
- **Branch & Merge Workflow:** Bureau Heads can create *branches* of the curriculum for major revisions (e.g. `NEP2020-Update`). Experts push changes to that branch's HEAD. When ready, a "merge request" triggers a workflow: the system creates a diff between branch HEAD and `main` , displays the changes, and allows the head of the bureau to approve. Upon merge, the backend performs a database transaction combining the branches: since each branch is just a sequence of snapshots, merging can be implemented by creating a new commit whose parent is the target branch HEAD, and whose content is the branch content. (Underlying MongoDB operations should be atomic; Spring Boot transactions or MongoDB's two-phase writes can ensure no partial merge.)

- **Two-Phase Commit & ACID:** For operations affecting multiple documents (e.g. merging branches, publishing curricula, adding new courses), we use database transactions or a two-phase commit (MongoDB 4.0+ multi-document transactions or equivalent). This guarantees that either all related records (curriculum doc, version snapshot, audit log) are saved, or none are – preventing half-applied changes.

3.3 AI-Powered NEP 2020 Compliance Engine

- **Credit Structure Validator:** The system continuously checks that total credits and component credits meet NEP norms. For example, UGC/NEP guidelines set a **4-year B.Tech minimum at 160 credits** including research credits [16†L1455-L1459] . The engine enforces this by summing `modules[].credits` and flagging errors if the limit is exceeded or if mandatory research credits (often 12 credits for a capstone) are missing [16†L1455-L1459] .
- **Mandatory Component Check:** NEP 2020 mandates courses like Universal Human Values (UHV), vocational training, and internships. The compliance UI shows visual indicators (e.g. green checkmarks) when required elements are present. For instance, if an internship module is included worth the required credits, it passes. NEP specifically calls for life-skills, ethics, and internships in higher education [33†L3465-L3473] , so our rules engine scans the curriculum JSON: if “Universal Human Values” or “Ethics” isn’t listed, it raises a live warning.
- **Bloom’s Taxonomy Analyzer:** Using NLP, we classify each learning outcome into Bloom’s cognitive levels (Remember, Understand, Apply, Analyze, Evaluate, Create). Research shows this can be automated: e.g., an SVM with data augmentation achieved ~94% accuracy classifying outcomes by Bloom level [29†L0-L4] [30†L0-L4] . We can implement a trained classifier (or fine-tune an LLM) to flag any outcome that is overly low or high-level for its intended course. For example, a 4th-year elective’s outcomes should include higher-order verbs (“design, evaluate”), and the engine can highlight if too many outcomes are mere “define/identify” tasks. This ensures course objectives progress in complexity.

3.4 Smart Syllabus Assistant & Resource Matcher

- **LLM Curriculum Draft Generator:** We incorporate a generative AI assistant (e.g. GPT-4 or a fine-tuned open model) that can suggest syllabus outlines. Given a course title or domain, it can propose modules, topics, and lab exercises. For example, prompting an LLM with “Design a syllabus outline for **Edge Computing** in an IoT course” might yield current subtopics (like “Fog architecture,” “Edge AI”). Several educators have experimented with using ChatGPT to create course outlines and curate resources [61†L143-L147] . CurriCraft can similarly call an LLM via API to generate drafts, which experts then refine.
- **Auto-OER Curator:** To enrich content, the system automatically finds Open Educational Resources matching each topic. We can use web APIs or search for Creative Commons materials (e.g. OpenStax textbooks, NPTEL course links, OER Commons) by keyword. Alternatively, leveraging the LLM itself: e.g. “List three free online resources (with links) for topic X.” The medium write-up by Kapadia shows ChatGPT can list courses and links when prompted with syllabus topics [61†L143-L147] . CurriCraft automates this: for each module name or keyword, it queries trusted OER databases or uses an AI to

retrieve relevant video lectures, textbooks, or article links, which are then attached to that module's `resources` array.

- **Industry Keyword Alignment:** The system can scan a corpus of job listings (scraped from places like Naukri or via LinkedIn's API) to detect trending technical skills. It then compares these keywords to the current curriculum topics. If a high-demand topic (e.g. "Quantum Computing" or "Edge AI") is missing, it prompts the expert to consider adding it. This might use basic NLP (TF-IDF or embeddings) to identify gaps. Though not trivial, this continuous market analysis ensures curricula stay industry-relevant.

3.5 Multi-Threaded Analytics & Impact Dashboard

- **Asynchronous Reporting:** Nationwide stats (e.g. adoption counts, average credits per curriculum) can be expensive to compute. We offload these to worker threads or processes. For instance, an endpoint `/analytics/adoption-stats` might trigger a background job (via BullMQ) that aggregates data across all curricula (often using MongoDB's aggregation pipeline or a separate OLAP store). The dashboard UI periodically polls for job completion.
- **Institutional Adoption Tracker:** The analytics dashboard includes interactive maps/charts showing how many institutes have adopted each model curriculum, by region or category. We can visualize, for example, heatmaps of states with highest adoption. Tools like Recharts/Chart.js render these graphs on the client.
- **Gap Analysis Reporting:** Another dashboard compares the "ideal" model curriculum to actual university syllabi (if available). For example, if the AICTE model includes 14 internship credits but many colleges only have 8, we highlight this misalignment. This uses the collected curriculum data vs. actual course data. Charts (bar charts, Sankey diagrams) display such gaps.

3.6 Hierarchical Approval & Governance Workflow

- **Multi-Tier Roles:** As noted, users are scoped by roles:
- **AICTE Admin:** can configure policies (e.g. set credit caps), onboard bureaus, and has final publish authority.
- **Bureau Head:** creates subject clusters (e.g. "Electrical Engineering"), assigns committees, reviews and approves merge requests for that domain.
- **Curriculum Expert:** (faculty, industry specialist) writes modules, responds to feedback, and updates drafts.
- **Public Viewer:** read-only role – can browse published curricula and download PDFs. These map to an RBAC scheme: roles inherit and escalate as needed (e.g. an Admin has all Bureau Head privileges plus global config rights `【54†L93-L101】`).
- **Inline Commenting & Review Threads:** Similar to Google Docs or pull-request reviews, users can select any text or outcome and leave a comment. The UI attaches these to that fragment in the document. Comments support `@mentions` to ping other users. Each comment thread has states (open/resolved). Technically, comments are tied to a document position (or learning outcome ID). The collaboration system broadcasts new comments via WebSocket to all collaborators on that doc.

A simple thread model in MongoDB stores `comments: [ {author, text, status, replies...} ]` linked to curriculum and possibly snapshot ID.

4. Key MongoDB Data Models

CurriCraft's flexible data models capture curricula and version history as JSON-like documents. For example:

```
// **Curriculum Model** (example)
{
  "_id": ObjectId("65a123..."),
  "title": "B.Tech Computer Science & Engineering",
  "code": "CSE-UG-2026",
  "degreeLevel": "Undergraduate",
  "totalCredits": 160,
  "status": "In_Review",
  "activeBranch": "main",
  "nepCompliance": {
    "uhvVerified": true,
    "internshipCredits": 14,
    "bloomTaxonomyScore": 0.88
  },
  "modules": [
    {
      "moduleId": "M101",
      "title": "Data Structures & Algorithms",
      "credits": 4,
      "lectureHours": 3,
      "practicalHours": 2,
      "learningOutcomes": [
        "Analyze time complexity using Big-0 notation",
        "Implement non-linear data structures"
      ],
      "resources": [
        { "title": "NPTEL DSA Course", "url": "https://nptel.ac.in/..." }
      ]
    }
    // ... more modules
  ],
  "version": "1.2.0",
  "updatedAt": "2026-08-06T20:25:29Z"
}
```

- **Curriculum Document:** This contains all modules for a degree, nested in one document. MongoDB's document model easily represents the hierarchical data (degree→modules→outcomes). Such a schema is schema-agile: new fields (like `nepCompliance`) can be added without rigid migrations.

Mongo's ability to store arrays/embedded docs makes queries like "find a module by ID" or "sum credits" straightforward.

```
// **Version Control Snapshot Model**
{
  "_id": ObjectId("65b456..."),
  "curriculumId": ObjectId("65a123..."),
  "commitHash": "a1b2c3d4e5",
  "parentHash": "f9e8d7c6b5",
  "author": ObjectId("user_99"),
  "commitMessage": "Added NEP 2020 Universal Human Values module",
  "snapshotData": { /* complete curriculum JSON at commit time */ },
  "createdAt": "2026-08-06T20:25:29Z"
}
```

- **Version Snapshot:** Each commit snapshot is a separate document containing a full copy of the curriculum at that point. Storing snapshots (rather than diffs) simplifies restoring old versions. The chain of `parentHash` links forms our commit history. When comparing versions, the backend retrieves two snapshots and computes diffs on `snapshotData`. Since each snapshot is self-contained, it's easy to publish or rollback to any commit atomically.

5. Primary API Endpoints

We define RESTful endpoints and a WebSocket channel for collaboration:

- **Curriculum & Collaboration:**

- `GET /api/v1/curriculum` – list curricula (filterable by program, domain).
- `POST /api/v1/curriculum` – create a new curriculum draft (requires auth).
- `GET /api/v1/curriculum/:id` – fetch a curriculum (latest draft or published).
- `GET /api/v1/curriculum/:id/diff?versionA=1.0.0&versionB=1.2.0` – return a JSON diff between two versions (server-side diff logic).
- `WS /ws/curriculum/:id` – WebSocket endpoint for real-time editing. Clients connect here to receive and send Yjs document updates and awareness events.

- **AI & Compliance:**

- `POST /api/v1/ai/generate-syllabus` – take a partial syllabus or course title and return AI-suggested modules/outcomes (calls LLM service).
- `POST /api/v1/compliance/verify` – submit a curriculum JSON; responds with compliance report (e.g. missing UHV, credit totals, Bloom scores).

- **Governance & Analytics:**

- `POST /api/v1/curriculum/:id/merge-request` – submit the current draft branch for approval (transition status, notify Bureau Head).
- `GET /api/v1/analytics/adoption-stats` – initiate/adopt the background job that computes nationwide adoption metrics (returns job ID or stats once ready).

Each protected endpoint checks the user's role (via JWT) against our RBAC policy [54†L93-L101] before performing the action.

6. SIH Pitch Differentiation Matrix

Compared to traditional manual processes, CurriCraft AI offers:

Metric	Traditional Baseline Solution	CurriCraft AI Solution
Document Collaboration	Sequential edit & re-upload via email/shares.	Real-time multi-user editing via WebSockets and CRDTs [57†L112-L120] .
Version Handling	Manual file naming (<code>v1_final_final.pdf</code>).	Automated Git-like commits and visual diff UI, tracking history systematically.
Policy Verification	Manual review by committees (time-consuming).	Automated NEP 2020 compliance engine (instant rules checks: credit caps, mandatory courses, etc. from NEP guidelines [33†L3465-L3473] [26†L21-L29]).
Performance	Synchronous, blocking processing (e.g. sync PDF gen).	Asynchronous multi-threaded workers (job queues) for heavy tasks [44†L52-L60] , keeping UI responsive.
Resource Mapping	Static, manual reference lists.	AI-driven OER/NPTEL search that dynamically curates open resources and videos for each topic (using LLMs and search).

Each of these differentiators is supported by modern best practices. For example, our collaborative editor draws on CRDT research to achieve **no-lock concurrency** [57†L112-L120] . Our RBAC-based governance aligns with industry security guides [54†L93-L101] . And our use of AI for content and compliance reflects the emerging trend of applying generative models and NLP to education [61†L143-L147] [30†L0-L4] . Together, these ensure CurriCraft AI significantly streamlines and modernizes AICTE's curriculum workflows.

Sources: The above design is informed by official NEP 2020 policy documents [26†L21-L29] [33†L3465-L3473] , AICTE curricular updates, and technical research on CRDT-based editing [57†L112-L120] [38†L917-L924] , AI/NLP in education [28†L51-L60] [61†L143-L147] , and software architecture best practices [42†L230-L239] [44†L52-L60] . These citations connect our proposed system to current standards and research.